

MARAN PMS — Motor Transaccional, Anulaciones y Blindaje de Cajas

PROMPT: Ledger unificado + política de reversión + integridad de turnos

DIAGNÓSTICO ACTUAL

Después de auditar el código, la situación real es:

Pagos dispersos en 5 tablas separadas:

- `payments` — reservas
- `event_payments` — eventos
- `event_table_payments` — mesas de eventos
- `spa_payments` — spa
- `admin_cash_movements` — caja administrativa

Anulaciones inconsistentes por módulo:

- `admin_cash_movements` → tiene `anulado: boolean + motivoAnulacion` 
- `charges` y `payments` de reservas → DELETE directo, sin campo de estado ni log 
- `event_payments` y `spa_payments` → DELETE directo, sin trazabilidad 
- `cash_movements` (caja recepción) → no tiene campo `anulado` 

Caja:

- `getOrCreateActiveTurno()` autocrea un turno si no hay ninguno abierto → correcto para disponibilidad
- `cash_movements` no tiene campo `anulado` → si se registró mal, solo se puede borrar
- No hay protección explícita en `registerCashMovement` contra agregar a un turno

cerrado (aunque `getOrCreateActiveTurno` busca uno abierto, lo cual previene el problema indirectamente)

CAMBIO 1 — Campo `status` en `charges` y `payments` de reservas

En lugar de borrar un cargo o pago, marcarlo como anulado con motivo y quién lo anuló.

A — Migración de DB

```
ALTER TABLE charges
  ADD COLUMN IF NOT EXISTS status      text      NOT NULL DEFAULT 'active',
  ADD COLUMN IF NOT EXISTS anulado_por text,
  ADD COLUMN IF NOT EXISTS motivo_anulacion text,
  ADD COLUMN IF NOT EXISTS anulado_at  timestamp;
```

```
ALTER TABLE payments
  ADD COLUMN IF NOT EXISTS status      text      NOT NULL DEFAULT 'active',
  ADD COLUMN IF NOT EXISTS anulado_por text,
  ADD COLUMN IF NOT EXISTS motivo_anulacion text,
  ADD COLUMN IF NOT EXISTS anulado_at  timestamp;
```

B — Schema Drizzle (`shared/schema.ts`)

```
// En charges:
status:      text("status").notNull().default("active"),
anuladoPor:  text("anulado_por"),
motivoAnulacion: text("motivo_anulacion"),
anuladoAt:   timestamp("anulado_at"),

// En payments:
status:      text("status").notNull().default("active"),
anuladoPor:  text("anulado_por"),
motivoAnulacion: text("motivo_anulacion"),
anuladoAt:   timestamp("anulado_at"),
```

C — Reemplazar DELETE por PATCH /anular en el backend

```
// — Anular cargo (reemplaza DELETE /api/charges/:id) —————
app.patch("/api/charges/:id/anular", requireAuth, async (req, res) => {
  try {
    const { motivoAnulacion, anuladoPor } = req.body;
```

```

    if (!motivoAnulacion?.trim()) {
        return res.status(400).json({ error: "El motivo de anulación es requerido"
    }

    const existing = await storage.getCharge(req.params.id);
    if (!existing) return res.status(404).json({ error: "Cargo no encontrado" });
    if (existing.status === "anulado") {
        return res.status(400).json({ error: "El cargo ya está anulado" });
    }

    // Mantener protección existente de reserva cerrada
    if (existing.reservationId) {
        const reservation = await storage.getReservation(existing.reservationId);
        if (reservation && isReservationLocked(reservation)) {
            return res.status(403).json({ error: "No se puede anular cargos de una re
        }
    }

    const updated = await db.update(charges)
        .set({
            status: "anulado",
            anuladoPor: anuladoPor || null,
            motivoAnulacion,
            anuladoAt: new Date(),
        })
        .where(eq(charges.id, req.params.id))
        .returning();

    res.json(updated[0]);
} catch (e: any) {
    res.status(500).json({ error: e.message });
}
});

// — Anular pago (reemplaza DELETE /api/payments/:id) —————
app.patch("/api/payments/:id/anular", requireAuth, async (req, res) => {
    try {
        const { motivoAnulacion, anuladoPor } = req.body;
        if (!motivoAnulacion?.trim()) {
            return res.status(400).json({ error: "El motivo de anulación es requerido"
        }

        const payResult = await db.execute(
            sql`SELECT * FROM payments WHERE id = ${req.params.id}`
        );
        const pay = payResult.rows?.[0] as any;
        if (!pay) return res.status(404).json({ error: "Pago no encontrado" });
    }

```

```

    if (pay.status === "anulado") {
      return res.status(400).json({ error: "El pago ya está anulado" });
    }

    if (pay.reservation_id) {
      const reservation = await storage.getReservation(pay.reservation_id);
      if (reservation && isReservationLocked(reservation)) {
        return res.status(403).json({ error: "No se puede anular pagos de una res"
      }
    }

    const updated = await db.execute(sql`
      UPDATE payments
      SET status = 'anulado',
          anulado_por = ${anuladoPor || null},
          motivo_anulacion = ${motivoAnulacion},
          anulado_at = NOW()
      WHERE id = ${req.params.id}
      RETURNING *
    `);

    res.json(updated.rows[0]);
  } catch (e: any) {
    res.status(500).json({ error: e.message });
  }
});

```

D — Mantener DELETE existente como fallback (no eliminar todavía)

Los endpoints `DELETE /api/charges/:id` y `DELETE /api/payments/:id` pueden quedar pero **agregar un log de advertencia** para detectar si algo los sigue llamando:

```

app.delete("/api/charges/:id", requireAuth, async (req, res) => {
  console.warn(`[DEPRECADO] DELETE /api/charges/${req.params.id} - usar PATCH /an
  // ... lógica existente sin cambios
});

```

E — Filtrar anulados en las consultas existentes

En todos los endpoints GET que devuelven cargos y pagos, agregar filtro:

```

// Al listar cargos de una reserva:
WHERE reservation_id = ${id} AND (status IS NULL OR status = 'active')

// En Drizzle:

```

```

.where(and(
  eq(charges.reservationId, reservationId),
  or(isNull(charges.status), eq(charges.status, "active"))
))

```

F — Modal de anulación en el frontend

Reemplazar el botón de eliminar cargo/pago por un botón “Anular” que abre un dialog:

```

// AnularDialog reutilizable
function AnularDialog({
  open,
  onClose,
  onConfirm,
  titulo,
  descripcion,
}: {
  open: boolean;
  onClose: () => void;
  onConfirm: (motivo: string) => void;
  titulo: string;
  descripcion?: string;
}) {
  const [motivo, setMotivo] = useState("");

  return (
    <Dialog open={open} onOpenChange={onClose}>
      <DialogContent className="max-w-md">
        <DialogHeader>
          <DialogTitle className="flex items-center gap-2 text-destructive">
            <AlertTriangle className="h-5 w-5" />
            {titulo}
          </DialogTitle>
          {descripcion && (
            <DialogDescription>{descripcion}</DialogDescription>
          )}
        </DialogHeader>
        <div className="space-y-3 py-2">
          <Label>Motivo de anulación <span className="text-destructive">*</span><
          <Textarea
            placeholder="Indicar el motivo..."
            value={motivo}
            onChange={(e) => setMotivo(e.target.value)}
            rows={3}
          />
        </div>
      </DialogContent>
    </Dialog>
  );
}

```

```

<DialogFooter>
  <Button variant="ghost" onClick={onClose}>Cancelar</Button>
  <Button
    variant="destructive"
    disabled={!motivo.trim()}
    onClick={() => { onConfirm(motivo); onClose(); }}
  >
    Confirmar Anulación
  </Button>
</DialogFooter>
</DialogContent>
</Dialog>
);
}

```

Usar este componente en el folio de reservas, lista de pagos, y cualquier lugar donde hoy hay un botón de eliminar cargo/pago.

Los registros anulados deben seguir visibles en el folio pero con estilo tachado y badge "ANULADO":

```

<TableRow
  key={charge.id}
  className={charge.status === "anulado" ? "opacity-50" : ""}
>
  <TableCell className={charge.status === "anulado" ? "line-through" : ""}>
    {charge.description}
  </TableCell>
  <TableCell>
    {charge.status === "anulado" ? (
      <Badge variant="destructive" className="text-xs">ANULADO</Badge>
    ) : null}
  </TableCell>
</TableRow>

```

CAMBIO 2 — Campo `anulado` en `event_payments` y `spa_payments`

```

ALTER TABLE event_payments
  ADD COLUMN IF NOT EXISTS status          text NOT NULL DEFAULT 'active',
  ADD COLUMN IF NOT EXISTS motivo_anulacion text,
  ADD COLUMN IF NOT EXISTS anulado_at      timestamp;

ALTER TABLE spa_payments

```

```
ADD COLUMN IF NOT EXISTS status          text NOT NULL DEFAULT 'active',
ADD COLUMN IF NOT EXISTS motivo_anulacion text,
ADD COLUMN IF NOT EXISTS anulado_at      timestamp;
```

Aplicar el mismo patrón PATCH /anular para:

- PATCH /api/events/:eventId/payments/:payId/anular
- PATCH /api/spa/payments/:id/anular

Protecciones equivalentes:

- Evento facturado (status === "invoiced") → no se puede anular
- Cuenta spa cerrada (status === "closed") → no se puede anular

CAMBIO 3 — Campo `anulado` en `cash_movements` de caja recepción

```
ALTER TABLE cash_movements
ADD COLUMN IF NOT EXISTS anulado          boolean NOT NULL DEFAULT false,
ADD COLUMN IF NOT EXISTS motivo_anulacion text,
ADD COLUMN IF NOT EXISTS anulado_por      text,
ADD COLUMN IF NOT EXISTS anulado_at      timestamp;
```

Schema Drizzle:

```
// En cashMovements:
anulado:          boolean("anulado").notNull().default(false),
motivoAnulacion:  text("motivo_anulacion"),
anuladoPor:       text("anulado_por"),
anuladoAt:        timestamp("anulado_at"),
```

Nuevo endpoint de anulación:

```
app.patch("/api/cash/movements/:id/anular", requireAuth, async (req, res) => {
  try {
    const { motivoAnulacion, anuladoPor } = req.body;
    if (!motivoAnulacion?.trim()) {
      return res.status(400).json({ error: "Motivo requerido" });
    }

    // Verificar que el turno asociado esté abierto
```

```

const movResult = await db.select().from(cashMovements)
  .where(eq(cashMovements.id, req.params.id));
const mov = movResult[0];
if (!mov) return res.status(404).json({ error: "Movimiento no encontrado" });
if (mov.anulado) return res.status(400).json({ error: "Ya está anulado" });

if (mov.shiftId) {
  const shiftResult = await db.select().from(cashShifts)
    .where(eq(cashShifts.id, mov.shiftId));
  const shift = shiftResult[0];
  if (shift && shift.status === "closed") {
    return res.status(403).json({
      error: "No se puede anular movimientos de un turno cerrado"
    });
  }
}

await db.update(cashMovements)
  .set({
    anulado: true,
    motivoAnulacion,
    anuladoPor: anuladoPor || null,
    anuladoAt: new Date(),
  })
  .where(eq(cashMovements.id, req.params.id));

res.json({ ok: true });
} catch (e: any) {
  res.status(500).json({ error: e.message });
}
});

```

Filtrar anulados en los cálculos del turno

En `closeShift()` y en `getCashSummary()`, excluir movimientos anulados de los totales:

```

// En closeShift - al calcular totals:
const movements = await db.select().from(cashMovements)
  .where(and(
    eq(cashMovements.shiftId, shiftId),
    eq(cashMovements.anulado, false) // ← agregar esta condición
  ));

// En getCashSummary - mismo filtro:
conditions.push(eq(cashMovements.anulado, false));

```

Mostrar anulados en la vista de turno con estilo diferenciado:

En el frontend de la caja, los movimientos anulados deben aparecer en la lista con:

- Texto tachado
 - Fondo rojizo suave
 - Badge "ANULADO" + motivo como tooltip
-

CAMBIO 4 — Vista de conciliación unificada (ledger por día)

No se propone crear una nueva tabla de ledger — sería sobreingeniería para este estadio. En cambio, crear **un endpoint de resumen diario unificado** que consolide los cobros de todos los módulos:

```
// GET /api/reports/caja-unificada?fecha=YYYY-MM-DD
app.get("/api/reports/caja-unificada", requireAuth, async (req, res) => {
  try {
    const fecha = req.query.fecha as string || getLocalToday();

    // 1. Pagos de reservas del día (no anulados)
    const pagosReservas = await db.execute(sql`
      SELECT 'reserva' AS origen, r.code AS referencia,
             g.first_name || ' ' || g.last_name AS nombre,
             p.method AS metodo, p.amount::numeric AS monto,
             p.date AS fecha
      FROM payments p
      LEFT JOIN reservations r ON r.id = p.reservation_id
      LEFT JOIN guests g ON g.id = r.guest_id
      WHERE p.date = ${fecha}
             AND (p.status IS NULL OR p.status = 'active')
    `);

    // 2. Pagos de eventos del día (no anulados)
    const pagosEventos = await db.execute(sql`
      SELECT 'evento' AS origen, e.name AS referencia,
             e.contact_name AS nombre,
             ep.method AS metodo, ep.amount::numeric AS monto,
             ep.paid_at::date AS fecha
      FROM event_payments ep
      LEFT JOIN events e ON e.id = ep.event_id
      WHERE ep.paid_at::date = ${fecha}
             AND (ep.status IS NULL OR ep.status = 'active')
    `);
```

```

// 3. Pagos de spa del día (no anulados)
const pagosSpa = await db.execute(sql`
    SELECT 'spa' AS origen, sa.id::text AS referencia,
           COALESCE(sc.name, 'Cliente') AS nombre,
           sp.method AS metodo, sp.amount::numeric AS monto,
           sp.created_at::date AS fecha
    FROM spa_payments sp
    LEFT JOIN spa_accounts sa ON sa.id = sp.account_id
    LEFT JOIN spa_clients sc ON sc.id = sa.client_id
    WHERE sp.created_at::date = ${fecha}
           AND (sp.status IS NULL OR sp.status = 'active')
`);

// Consolidar y agrupar por método de pago
const todos = [
    ...pagosReservas.rows,
    ...pagosEventos.rows,
    ...pagosSpa.rows,
];

const porMetodo: Record<string, number> = {};
for (const p of todos) {
    const m = (p as any).metodo || "otros";
    porMetodo[m] = (porMetodo[m] || 0) + parseFloat((p as any).monto || "0");
}

res.json({
    fecha,
    movimientos: todos,
    totalPorMetodo: porMetodo,
    totalGeneral: todos.reduce((s, p) => s + parseFloat((p as any).monto || "0"
));
} catch (e: any) {
    res.status(500).json({ error: e.message });
}
});

```

Este endpoint alimenta el **panel de conciliación diaria** en Administración, que en el futuro puede cruzarse con el cierre de turno de recepción para detectar diferencias.

NOTAS DE IMPLEMENTACIÓN

1. Orden de ejecución recomendado:

- Primero las migraciones de DB (ALTERs) — son aditivas, no rompen nada
- Luego los nuevos endpoints PATCH /anular
- Luego los filtros en los GETs (agregar `AND status = 'active'`)
- Luego el frontend (reemplazar botones delete por AnularDialog)
- Por último el endpoint de caja unificada

2. **Los DELETE existentes no se eliminan en esta fase** — se deprecian con `console.warn` para detectar si algún lugar del frontend los sigue llamando.
3. `isReservationLocked` **permanece igual** — sigue siendo la primera línea de defensa para reservas cerradas. La anulación con motivo es la segunda capa para reservas abiertas.
4. **Los cierres de turno ya calculados no se modifican** — el filtro de `anulado = false` en `closeShift()` solo afecta futuros cierres.
5. **El `AnularDialog` es un componente reutilizable** — se usa en folio de reservas, lista de pagos de eventos, cuentas de spa, y vista de caja.
6. **Principio rector de todo este cambio:**

Nada importante se borra. Todo se anula con motivo, operador y timestamp. Los registros anulados permanecen visibles pero excluidos de todos los cálculos.